WX·RADIO

RETRO RADIO WEATHER APP

Midterm submission · FA26-SDET373103 Web App Development for Mobile · October 2026

Expo app + Back4App backend: logins and sign-ups, saved cities, weather forecasts, and push weather alerts sent from Back4App Cloud Code.

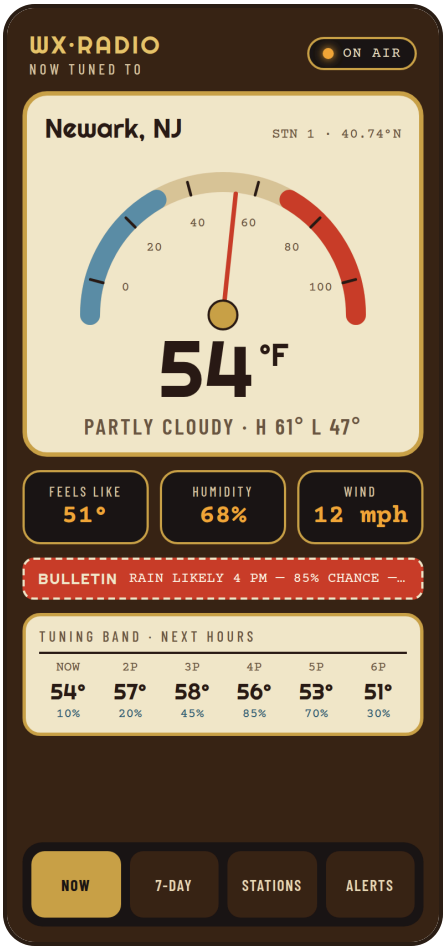
1. Design (made in Claude Design)
2. App documentation
3. API reference
4. Prompt log

Claude Design canvas: claude.ai/artifact/H43byitsjRMZkv4VnYWy7r
Code: github.com/AjHy2003/AjHy2003.github.io → `weather-app/`

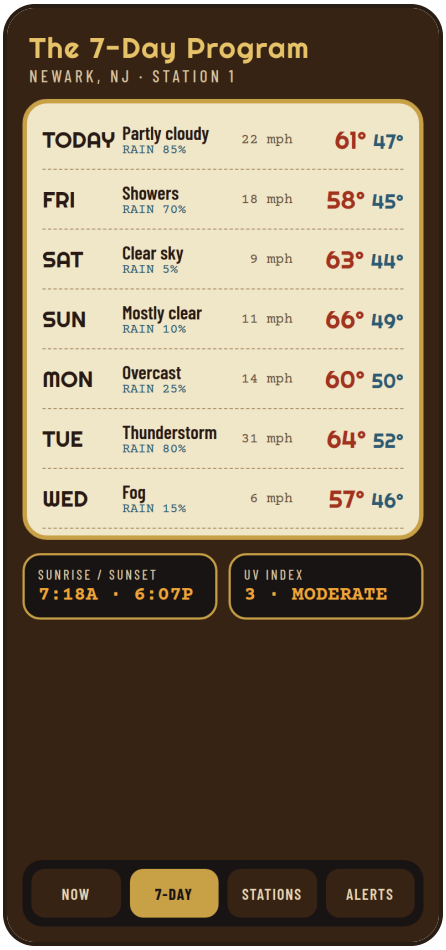
1 · Design — Claude Design

Theme: "Weather Station Radio." A 1940s tabletop radio with a walnut cabinet, cream dial face and brass trim. The temperature is an analog gauge with a needle, saved cities are radio "stations", and alerts come in as an "Emergency Broadcast" on red ticker tape.

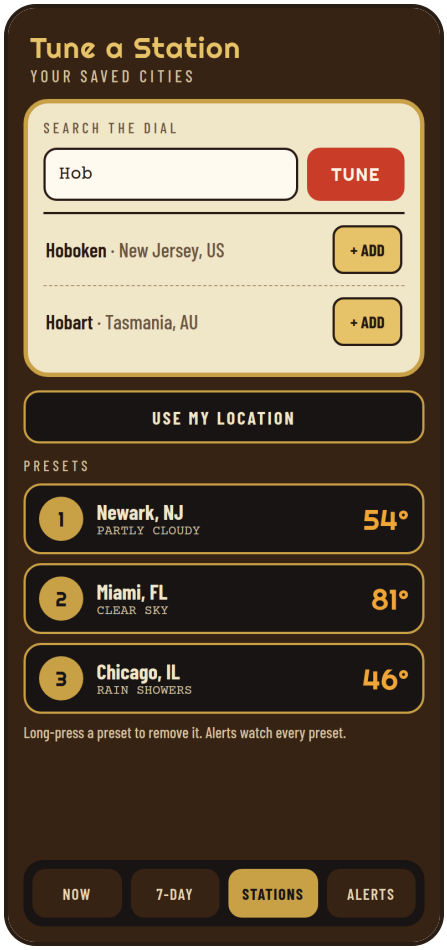
Designed in Claude Design: 5 phone screens (390 × 844) plus a style guide. The source files are in `weather-app/design/`.



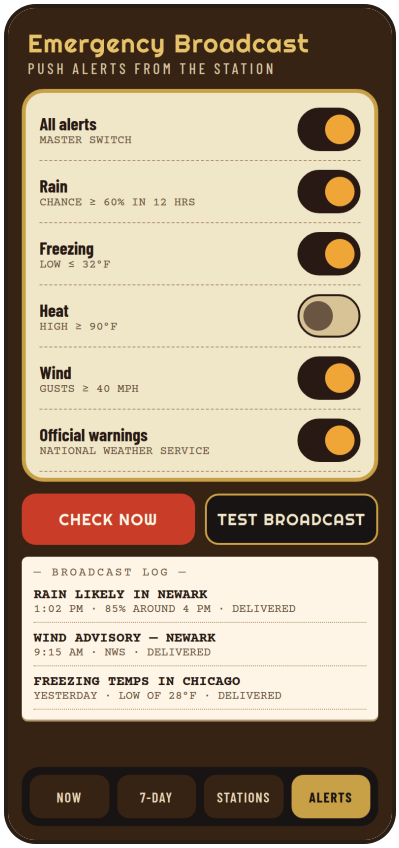
01 · NOW (HOME)



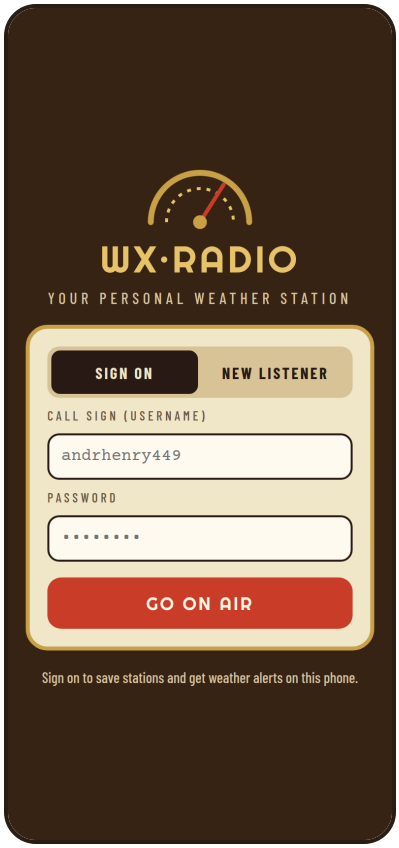
02 · 7-DAY



03 · STATIONS



04 · ALERTS + LOG



05 · SIGN ON / SIGN UP

Style guide

Weather Station Radio — style guide

A 1940s tabletop radio: walnut cabinet, cream dial face, brass trim, an analog needle and ticker-tape bulletins.

Walnut	Bakelite	Dial cream	Brass	Marquee gold	Tube amber	Signal red	Cold steel
#3b2416 · cabinet	#1c1714 · gauges, nav	#f3e6c8 · dial face	#c9a24a · trim, selected	#e9c46a · titles	#f2a93b · readouts	#c8402a · alerts, CTA	#2f5d73 · lows, rain

DISPLAY · RIGHTEOUS

54°F

Temperatures, screen titles, city names, big buttons.

LABELS · BARLOW CONDENSED

FEELS LIKE · HUMIDITY

Uppercase with wide tracking, like stamped panel labels.

READOUTS · COURIER PRIME

RAIN LIKELY 4 PM

Gauge readouts, ticker-tape bulletins, the broadcast log.

PRIMARY · GO ON AIR

SECONDARY · TEST

TAB · SELECTED

BULLETIN WIND ADVISORY UNTIL 8 PM

2 · App documentation

Web App Development for Mobile — Midterm.

See also: **PROMPTS.md** (prompts given to Claude) and **API.md** (API reference).

1. App brief

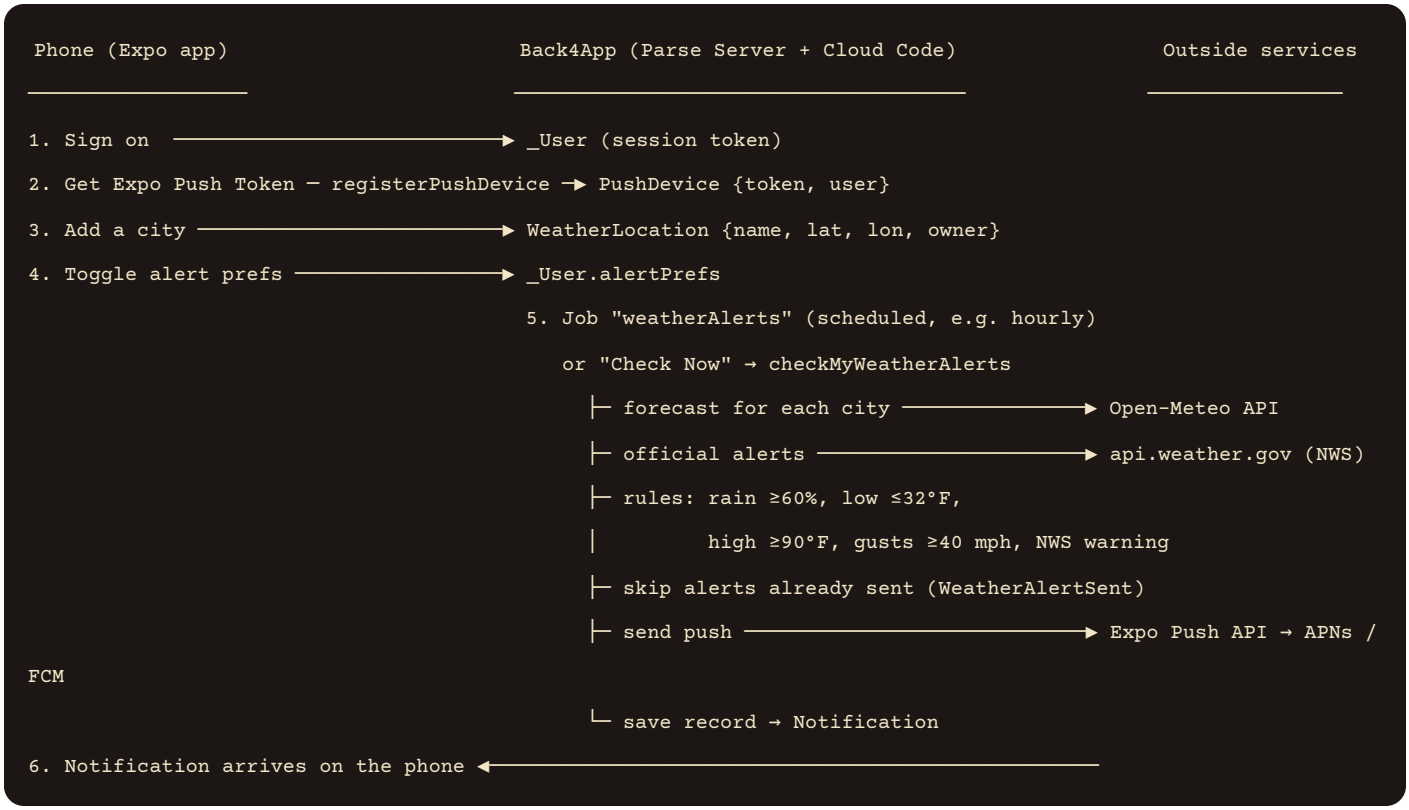
> Build a **full Expo app** that is a **weather app** with a **unique theme** designed in **Claude Design** (not the default weather look). It must have **logins and sign-ups**, and **push notifications for weather alerts sent from the backend using Back4App**.

Theme chosen: *Retro radio / weather-station dashboard.* A 1940s tabletop radio with a walnut cabinet, cream dial face and brass trim. The temperature is an analog gauge with a needle, saved cities are radio "stations" and "presets", and alerts arrive as an "Emergency Broadcast" on red ticker tape.

2. Features

Screen	What it does
Sign On	Log in, or create an account ("New Listener") with a username, password and optional email. Accounts are stored in Back4App's `_User` table.
Now	Analog temperature gauge, current conditions, feels-like / humidity / wind readouts, a bulletin bar (official NWS alert or rain warning), and a scrollable 12-hour "tuning band". Pull down to refresh.
7-Day	Daily forecast: conditions, rain chance, wind gusts, high/low, plus sunrise/sunset and UV index.
Stations	Search any city (Open-Meteo geocoding), add it as a preset, or **Use my location** (GPS). Tap to tune in; long-press to remove. Saved per user in Back4App.
Alerts	Inbox of every notification received on the phone (tap one, or tap a banner, to read the full message). Toggles for which alerts you want (master switch, rain, freezing, heat, wind, official NWS warnings), saved to your Back4App user. **Check Now** runs the backend alert check, and **Test Alert** sends a sample push. Also shows a broadcast log of every notification sent, and lets you sign off.

3. How push weather alerts work (backend)



Back4App classes

Class	Fields	Who can read it
`_User`	username, password (hashed), email, `alertPrefs`	the user
`WeatherLocation`	name, region, country, latitude, longitude, owner	owner only (ACL)
`PushDevice`	token, platform, deviceName, user	Cloud Code only
`WeatherAlertSent`	user, location, key (for example `rain:2026-10-22`)	Cloud Code only
`Notification`	title, message, sender, target, devices, sent, status	sender; admins via Cloud Code

Cloud Code (`cloud/main.js`)

Function / job	Purpose
`registerPushDevice`, `unregisterPushDevice`	Save or remove this phone's Expo Push Token
`checkMyWeatherAlerts({ force })`	Check the logged-in user's cities now and push any alerts
`sendTestWeatherAlert`	Push the current temperature for the first city (demo)
job `weatherAlerts`	Scheduled check for **every** user, which sends the alerts automatically
`sendPushNotification`, `broadcastPushNotification`	Manual sends (used by `push.html`, the admin web sender)

Function / job	Purpose
`getPushReceipts`	Checks with the Expo Push Receipts API that Apple/Google delivered a notification
`getNotificationHistory`	Broadcast log

4. APIs used

- **Open-Meteo** forecast and geocoding: free, no API key
- **National Weather Service** active alerts (`api.weather.gov`): free, no key, US locations only
- **Back4App REST API**: auth, data and Cloud Functions
- **Expo Push API**: delivery to iPhone (APNs) and Android (FCM)
- **Expo Push Receipts API**: confirms Apple/Google actually delivered each notification

5. Tech

Expo SDK 57 (React Native 0.86), `expo-notifications`, `expo-location`, `expo-font` with the Google Fonts Righteous, Barlow Condensed and Courier Prime, and `react-native-svg` for the gauge. Back4App is called through its REST API with `fetch`, because the Parse JS SDK doesn't bundle in Expo Go.

6. Design

Made in **Claude Design**. The files are in `design/`: `Main`, `Forecast`, `Stations`, `Alerts`, `SignOn` and `StyleGuide` (`.dc.html`) plus `canvas.json`.

Token	Hex	Use
Walnut	`#3b2416`	cabinet background
Bakelite	`#1c1714`	readouts, nav
Dial cream	`#f3e6c8`	panels
Brass	`#c9a24a`	trim, selected tab
Tube amber	`#f2a93b`	readout numbers
Signal red	`#c8402a`	alerts, main buttons
Cold steel	`#2f5d73`	lows, rain

3 · API reference

WX Radio uses **3 public weather APIs**, **the Back4App API** (including our own Cloud Function API) and **the Expo Push API**.

#	API	Used for	Auth	Code
1	Open-Meteo Forecast	Current, hourly, 7-day weather	none (free)	`src/weather.js` → `getForecast()`; `cloud/main.js` → `fetchForecast()`
2	Open-Meteo Geocoding	City search	none (free)	`src/weather.js` → `searchCities()`
3	National Weather Service (NWS)	Official alerts/warnings (US)	none (needs a User-Agent)	`src/weather.js` → `getOfficialAlerts()`; `cloud/main.js` → `fetchOfficialAlerts()`
4	Back4App (Parse) REST API	Sign up, login, saved cities, alert prefs	App ID + JavaScript Key (+ session token)	`src/back4app.js`
5	Back4App Cloud Functions (our API)	Push devices, weather alerts, test alert, history	same	`cloud/main.js`
6	Expo Push API	Deliver notifications to iPhone/Android	none (Expo Push Token)	`cloud/main.js` → `deliver()`
7	Expo Push Receipts API	Confirm Apple (APNs) / Google (FCM) delivered each notification	none (ticket ids)	`cloud/main.js` → `fetchReceipts()`, `getPushReceipts`

1. Open-Meteo Forecast API

`GET https://api.open-meteo.com/v1/forecast`

Example request (Newark, NJ):

```
https://api.open-meteo.com/v1/forecast?latitude=40.7357&longitude=-74.1724

&current=temperature_2m,apparent_temperature,relative_humidity_2m,weather_code,wind_speed_10m,wind_gusts_10m,is_day,precipitation
  &hourly=temperature_2m,precipitation_probability,weather_code,wind_gusts_10m,is_day

&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max,wind_gusts_10m_max,sunrise,sunset,uv_index_max
  &temperature_unit=fahrenheit&wind_speed_unit=mph&timezone=auto&forecast_days=7
```

Response (shortened):

```
{
  "current": { "time": "2026-10-22T13:15", "temperature_2m": 54.2, "apparent_temperature": 51,
               "relative_humidity_2m": 68, "weather_code": 2, "wind_speed_10m": 12 },
  "hourly":  { "time": ["2026-10-22T13:00", "..."], "temperature_2m": [54, "..."], "precipitation_probability":
[10, "..."] },
  "daily":   { "time": ["2026-10-22", "..."], "temperature_2m_max": [61, "..."], "temperature_2m_min": [47,
"..."] }
}
```

`weather_code` is a WMO code, which `describe()` in `src/weather.js` turns into "Partly cloudy", "Rain" and so on.

2. Open-Meteo Geocoding API

`GET https://geocoding-api.open-meteo.com/v1/search?name=Newark&count=8&language=en&format=json`

```
{ "results": [ { "id": 5101798, "name": "Newark", "admin1": "New Jersey", "country_code": "US",
                 "latitude": 40.73566, "longitude": -74.17237, "timezone": "America/New_York" } ] }
```

3. National Weather Service — active alerts

`GET https://api.weather.gov/alerts/active?point=40.7357,-74.1724` Headers: `User-Agent: WeatherAlertsMidterm (github.com/AjHy2003)`, `Accept: application/geo+json`

```
{ "features": [ { "properties": {
    "id": "urn:oid:2.49.0.1.840.0....", "event": "Wind Advisory", "severity": "Moderate",
    "headline": "Wind Advisory issued October 22 at 9:15AM EDT until October 22 at 8:00PM EDT" } } ] }
```

Alerts with severity **Moderate, Severe or Extreme** become push notifications.

4. Back4App REST API

Base URL `https://parseapi.back4app.com`. Every request sends:

```
X-Parse-Application-Id: <Application ID>
X-Parse-Javascript-Key: <JavaScript Key>
X-Parse-Session-Token: <token>     (after login)
Content-Type: application/json
```

Action	Request
Sign up	`POST /users` `{"username","password","email","alertPrefs"}` → `{"objectId","sessionToken"}`
Log in	`POST /login` `{"username","password"}` → user + `sessionToken`
Current user	`GET /users/me`
Log out	`POST /logout`
Save alert settings	`PUT /users/<objectId>` `{"alertPrefs":{"enabled":true,"rain":true,...}}`
List my cities	`GET /classes/WeatherLocation?where={"owner":{"__type":"Pointer","className":"_User","objectId":"<id>"}}`
Add a city	`POST /classes/WeatherLocation` `{"name","region","latitude","longitude","owner","ACL"}`
Remove a city	`DELETE /classes/WeatherLocation/<objectId>`

5. Our API — Back4App Cloud Functions (`cloud/main.js`)

Called with `POST https://parseapi.back4app.com/functions/<name>`, using the same headers.

Function	Body	Returns
`registerPushDevice`	`{"token":"ExponentPushToken[...]","platform":"ios","deviceName":"iPhone"}`	`{"objectId"}`
`unregisterPushDevice`	`{"token"}`	`{"removed":1}`
`checkMyWeatherAlerts`	`{"force":false}`	`{"locations":1,"found":[{"title":"🌧️ Rain likely in Newark",...}],"sent":1,"skipped":0}`
`sendTestWeatherAlert`	`{}`	`{"sent":1,"failed":[],"notificationId":"..."}`
`getPushReceipts`	`{"notificationId":"..."}`	`{"receiptStatus":"delivered","delivered":1,"pending":0,"errors":[]}`
`getNotificationHistory`	`{"limit":20}`	`[{"title","message","status","createdAt",...}]`
`sendPushNotification`	`{"title","message"}`	`{"sent","failed"}` (own devices)
`broadcastPushNotification` *(admin)*	`{"title","message","username?"}`	`{"sent","failed"}` (everyone or one user)
Job `weatherAlerts`	(scheduled hourly)	checks every user's cities and sends alerts

Try it from PowerShell (after logging in, use your session token):

```
$h = @{ "X-Parse-Application-Id"="<APP_ID>"; "X-Parse-Javascript-Key"="<JS_KEY>"; "X-Parse-Session-Token"="<SESSION>"; "Content-Type"="application/json" }
Invoke-RestMethod -Method Post https://parseapi.back4app.com/functions/sendTestWeatherAlert -Headers $h -Body "{}"
```

6. Expo Push API

`POST https://exp.host/--/api/v2/push/send` (sent by Cloud Code, never by the phone)

```
[ { "to": "ExponentPushToken[xxxxxxxxxxxxxxxxxxxxxx]", "title": "🌧️ Rain likely in Newark",
    "subtitle": "WX Radio · Newark", "priority": "high", "channelId": "default",
    "body": "85% chance of rain around 4 PM. Grab an umbrella!", "sound": "default",
    "data": { "type": "weather", "alert": "rain" } } ]
```

Response: `{ "data": [ { "status": "ok", "id": "..." } ] }`. Tokens that come back as `DeviceNotRegistered` are deleted from `PushDevice`.

7. Expo Push Receipts API

`POST https://exp.host/--/api/v2/push/getReceipts` (from Cloud Code, a few seconds after sending)

The send call above only says Expo **accepted** the message. The receipt says whether **Apple (APNs) / Google (FCM) delivered** it.

```
{ "ids": ["XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"] }
```

Response:

```
{ "data": { "XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX": { "status": "ok" } } }
```

or, on failure:

```
{ "data": { "XXXXXXXX-...": { "status": "error", "message": "...", "details": { "error": "DeviceNotRegistered" } } } }
```

Our Cloud Function `getPushReceipts` saves the result on the `Notification` row (`receiptStatus`: delivered / error / pending, plus `receiptErrors`). The app's **Test Alert** button and `wx-broadcast.html` check it automatically about 5 seconds after sending.

Receipt error	Meaning / fix
`DeviceNotRegistered`	The app was removed or the token changed. Sign off and sign back on.
`InvalidCredentials`	APNs/FCM keys in EAS are missing or wrong. Run `npx eas-cli@latest credentials`.
`MessageTooBig`	The payload is over 4 KB.
`MessageRateExceeded`	Too many messages to one device. Slow down.

4 · Prompt log

The prompts I gave Claude (Claude Code + Claude Design) to build WX Radio, in order. Only the prompts that defined the app or its features are listed; small spelling fixes only.

WX Radio (midterm)

#	Prompt	Result
1	Build a weather app as a full Expo app. Use Claude Design for a unique theme (not the default weather look). It must use logins and sign-ups, and push notifications for weather alerts from the backend using Back4App.	Plan: Expo app + Back4App login and push backend + weather alert rules
2	Theme: **retro radio / dashboard**	Claude Design canvas: 5 phone screens (Now, 7-Day, Stations, Alerts, Sign On) + style guide; app built to match
3	I need APIs as well.	Open-Meteo forecast and geocoding, National Weather Service alerts, Back4App REST + Cloud Functions, Expo Push; documented in `API.md`
4	We need test notifications, and an HTML page where I can send random notifications.	**Test Alert** button in the app; `wx-broadcast.html` web sender with weather presets and a random alert generator
5	It does not send the full notification to my Apple device, let's fix that. I think we need an API for the notifications.	Expo Push Receipts API to confirm Apple delivered each alert; high-priority pushes with an iPhone subtitle; tap a notification to read the full message; in-app inbox
6	Turn the Claude Design and the documentation into a PDF.	`WX-Radio-Midterm.pdf`

Backend setup (reused by WX Radio)

These prompts built the Expo + Back4App login and push-notification backend that WX Radio runs on.

#	Prompt	Result
1	Make a standard Expo app that uses Back4App with Expo Go dev. You can code it on GitHub and use CLIs.	Expo SDK 57 app on GitHub, Back4App via its REST API (works in Expo Go)
2	Push notifications (class steps): add `expo-notifications` and get the Expo Push Token for each device; save the token in a Back4App `PushDevice` class for each logged-in user; create a Back4App Cloud Code function that takes an Expo Push Token, title, message and optional data and sends it to the Expo Push API; set up Apple/Android push credentials in Expo/EAS; test on a real phone.	Login/sign-up, `PushDevice` class, `registerPushDevice` and `sendPushNotification` Cloud Functions, EAS + Firebase FCM V1 credentials
3	Make a webpage that I can type in that can send to the app.	Web sender page + admin-only `broadcastPushNotification` Cloud Function
4	Have every notification sent saved on the Back4App app.	`Notification` class logs every send (sender, target, delivered count, status) + `getNotificationHistory`